Revisiting Bruck: Phase-Efficient All-to-All Collective Communication in Reconfigurable Networks

Anton Juerss and Stefan Schmid

weizenbaum institut

Technische Universität Berlin

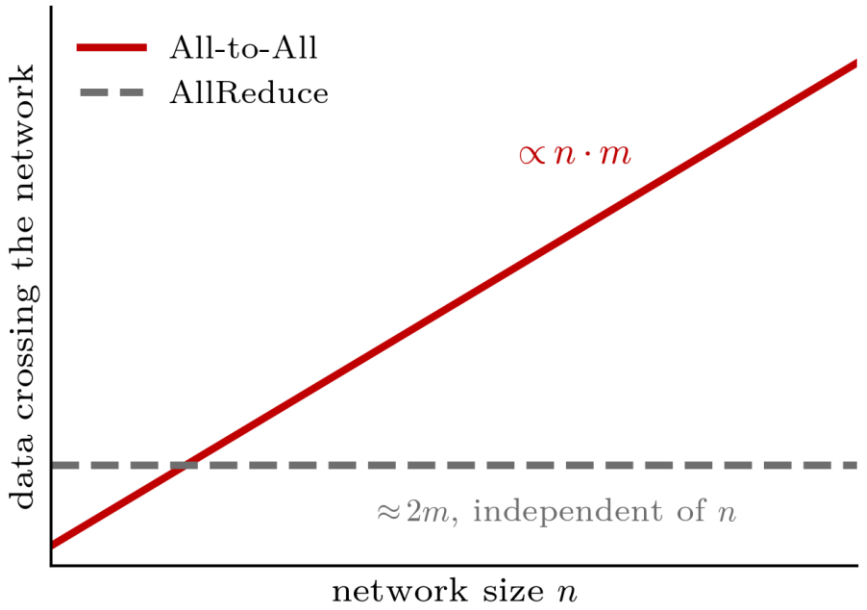
All-to-All: everyone sends to everyone

- Training and inference require synchronizations across accelerators
- Example: All-to-All — each node sends a unique message to every other node

All-to-All: everyone sends to everyone

- Training and inference require synchronizations across accelerators
- Example: All-to-All — each node sends a unique message to every other node

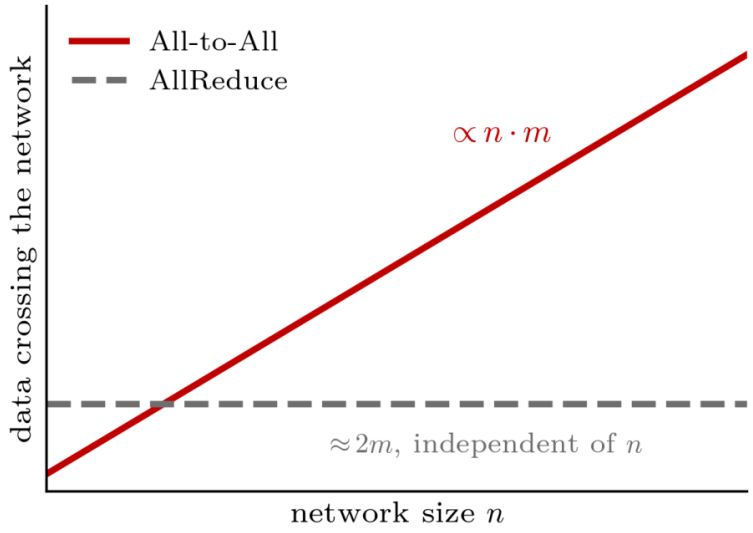
All-to-All cost grow linearly with n in rings topologies with network size



All-to-All: everyone sends to everyone

- Training and inference require synchronizations across accelerators
- Example: All-to-All — each node sends a unique message to every other node

All-to-All cost grow linearly with n in rings topologies with network size



All-to-All cost grow linearly with n in rings topologies with network size

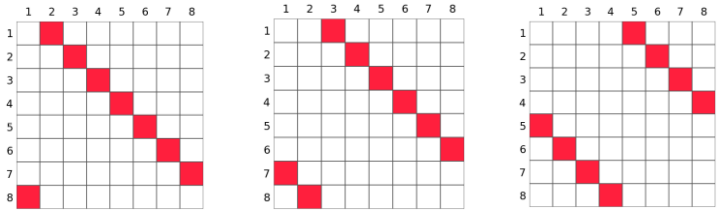
Can we use reconfigurable networks to fix this problem?

Collectives match reconfigurable networks

Collectives naturally offer:

- Known demand matrices
- Node synchronization between phases
- Concurrent transmissions have equal cost

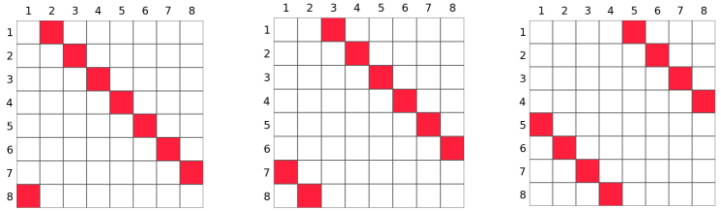
Natural opportunities to reconfigure



Collectives match reconfigurable networks

Collectives naturally offer:

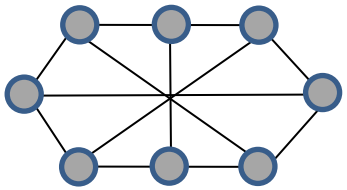
- Known demand matrices
- Node synchronization between phases
- Concurrent transmissions have equal cost



Natural opportunities to reconfigure

Common reconfiguration approaches:

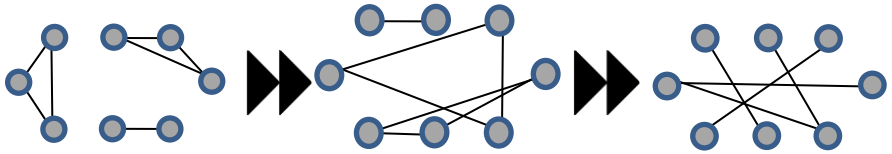
One-shot: before collective operations



K=0,1,2

,...

Dynamic: between each collective phases



K=0 K=1 K=2

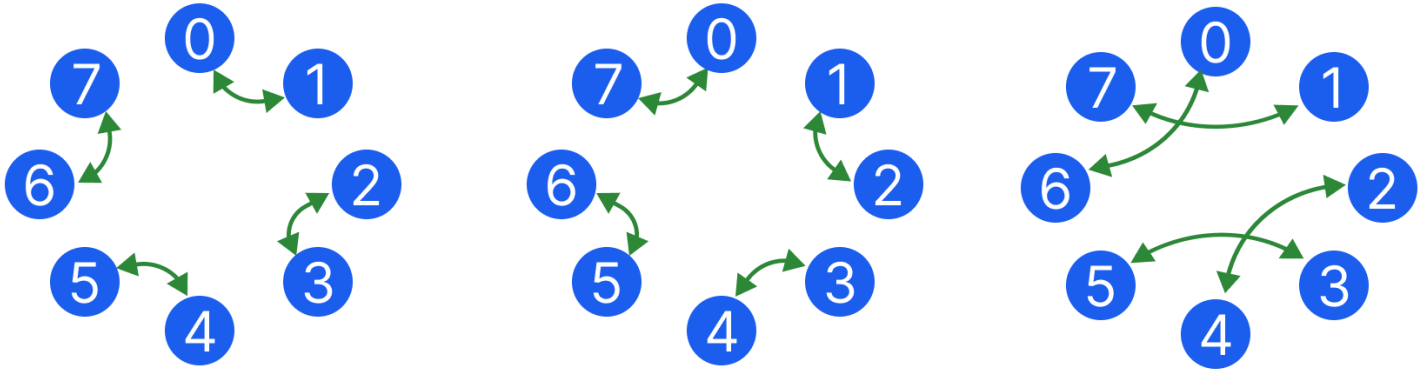
Optimization problem

Reconfiguration is global: every node waits for a delay δ (µs … ms)

Optimization problem

Reconfiguration is global: every node waits for a delay δ (µs … ms)

1 Matching: no connected topology + every-step reconfigurations



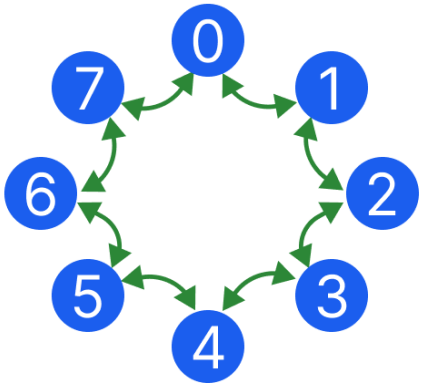
Optimization problem

Reconfiguration is global: every node waits for a delay δ (µs … ms)

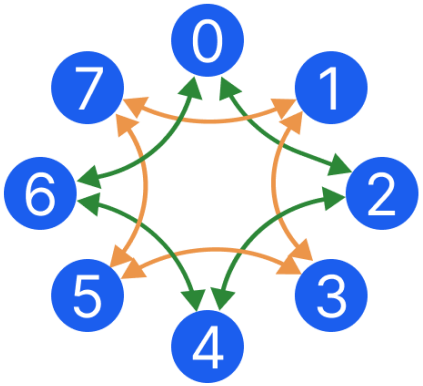
1 Matching: no connected topology + every-step reconfigurations

2 Matchings: OCS with $2n$ ports = every node keeps degree two

⇒ feasible topologies are rings



Ring topology



Small subrings

Optimization problem

Reconfiguration is global: every node waits for a delay δ (µs … ms)

1 Matching: no connected topology + every-step reconfigurations

2 Matchings: OCS with $2n$ ports = every node keeps degree two

⇒ feasible topologies are rings

Following number of reconfigurations:

1 Matching with s phases ⇒ $s-1$ reconfigurations with $(s-1)\cdot\delta$

2 Matchings ⇒ reusable topology states

number of reconfigurations $\in \{0,\dots,s-1\}$

Observation: 1. fewer phases require fewer reconfigurations

2. reusable reconfiguration states ⇒ δ amortized better

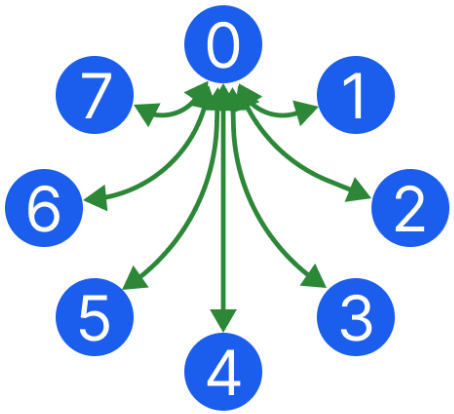
Challenges in reconfigurable All-to-All

Pairwise A2A: each node sends block directly to its destination

Challenges in reconfigurable All-to-All

Pairwise A2A: each node sends block directly to its destination

Option 1: one step with all simultaneous communication



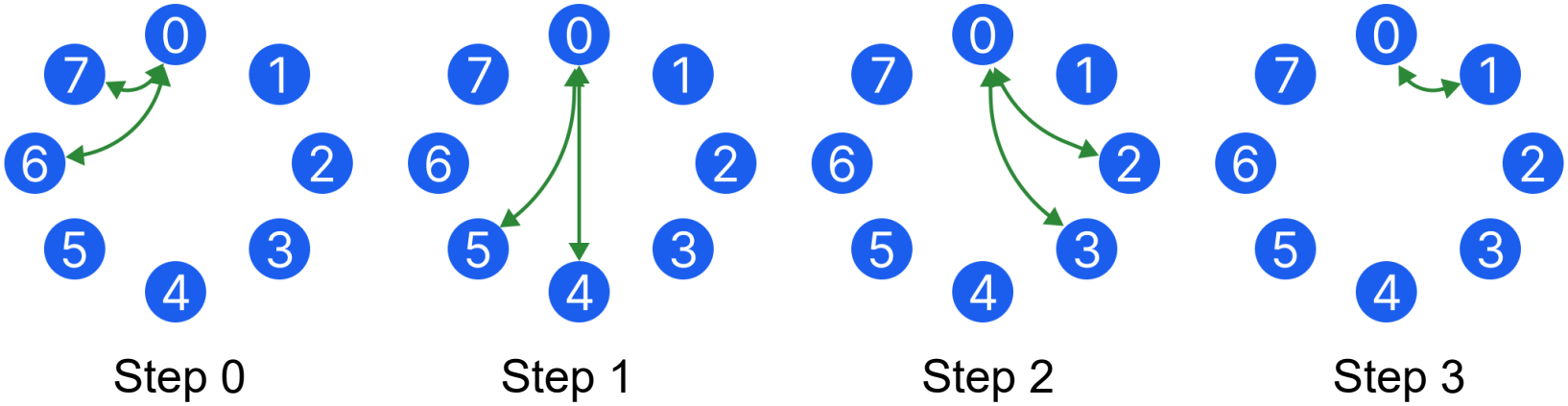
Transmissions from node 0

Challenges in reconfigurable All-to-All

Pairwise A2A: each node sends block directly to its destination

Option 1: one step with all simultaneous communication

Option 2: $\frac{n}{2}$ steps with single block transmissions



Challenges in reconfigurable All-to-All

Pairwise A2A: each node sends block directly to its destination

Option 1: one step with all simultaneous communication

Option 2: $\frac{n}{2}$ steps single block transmission



Observation: Both implementations are not feasible for reconfigurable networks

Bruck‘s algorithm for reconfigurable networks

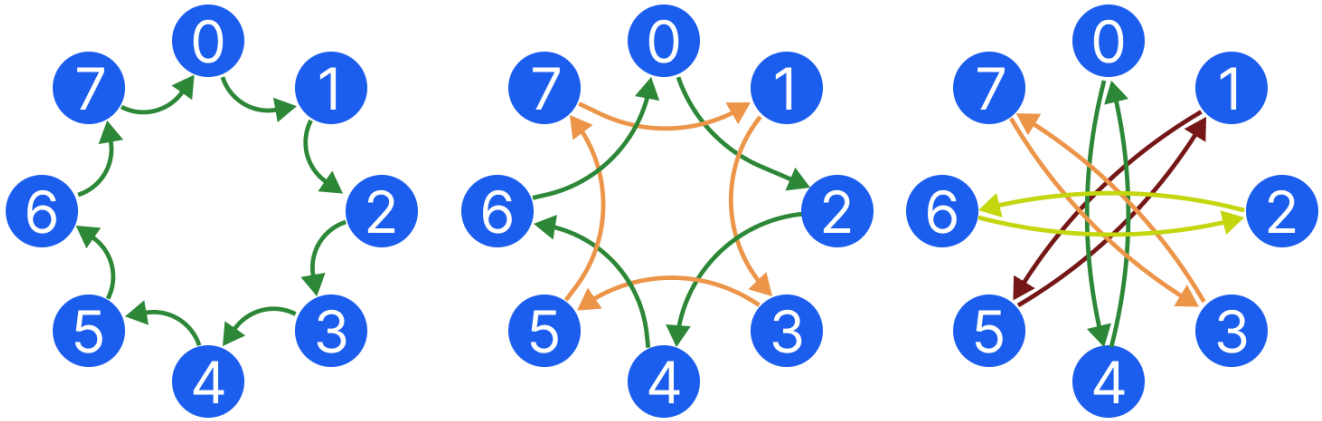
- Bruck et al. (1994): All-to-All in $\log_2 n$ phases by aggregating blocks

Bruck‘s algorithm for reconfigurable networks

- Bruck et al. (1994): All-to-All in $\log_2 n$ phases by aggregating blocks
- Built for a multiport message-passing model, not for reconfigurable networks

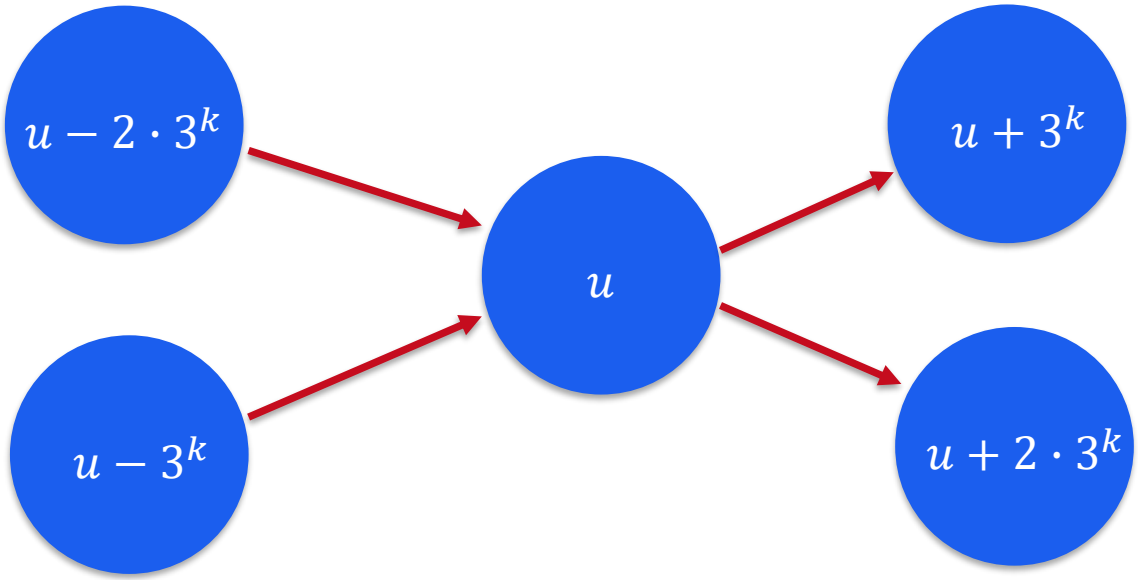
Bruck‘s algorithm for reconfigurable networks

- Bruck et al. (1994): All-to-All in $\log_2 n$ phases by aggregating blocks
- Built for a multiport message-passing model, not for reconfigurable networks
- Phase k: node u sends to $u + 2^k \Rightarrow$ one logical direction only



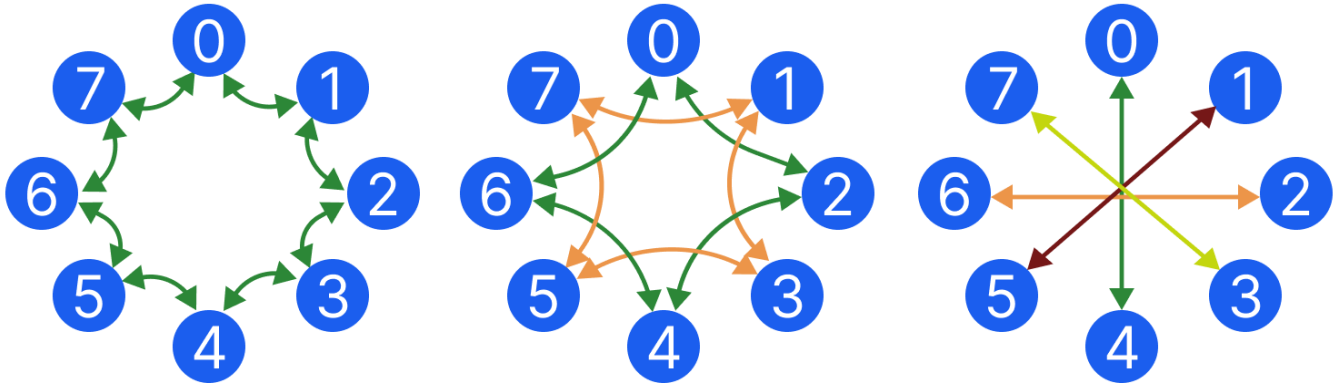
Bruck‘s algorithm for reconfigurable networks

- Bruck et al. (1994): All-to-All in $\log_2 n$ phases by aggregating blocks
- Built for a multiport message-passing model, not for reconfigurable networks
- Phase k: node u sends to $u + 2^k$ ⇒ one logical direction only
- With $2n$ ports: 2 sends + 2 receives ⇒ would require four links per node



Bruck‘s algorithm for reconfigurable networks

- Bruck et al. (1994): All-to-All in $\log_2 n$ phases by aggregating blocks
- Built for a multiport message-passing model, not for reconfigurable networks
- Phase k: node u sends to $u + 2^k \Rightarrow$ one logical direction only
- With $2n$ ports: 2 sends + 2 receives ⇒ would require four links per node
- Mirroring Bruck restores feasibility — but still $\log_2 n$ phases



ReTri: Reconfigurable Trivance

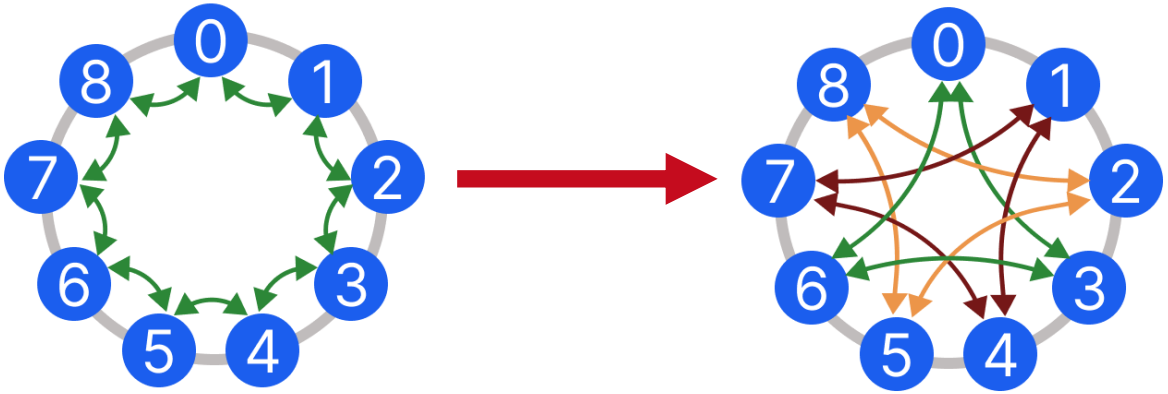
18th August SIGCOMM
1:50 PM Research Session 3

- Recently proposed algorithm to reduce phases ⇒ Trivance
- Phase k: node u sends to $u + 3^k$ and $u - 3^k$

ReTri: Reconfigurable Trivance

- Recently proposed algorithm to reduce phases ⇒ Trivance
- Phase k: node u sends to $u + 3^k$ and $u - 3^k$
- Two peers per phase ⇒ matches the two optical links per node
- Symmetric exchange ⇒ both directions per optical link are used
- All-to-All completes in $\log_3 n$ communication phases

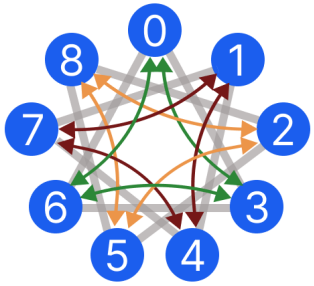
Case: Topology remains static



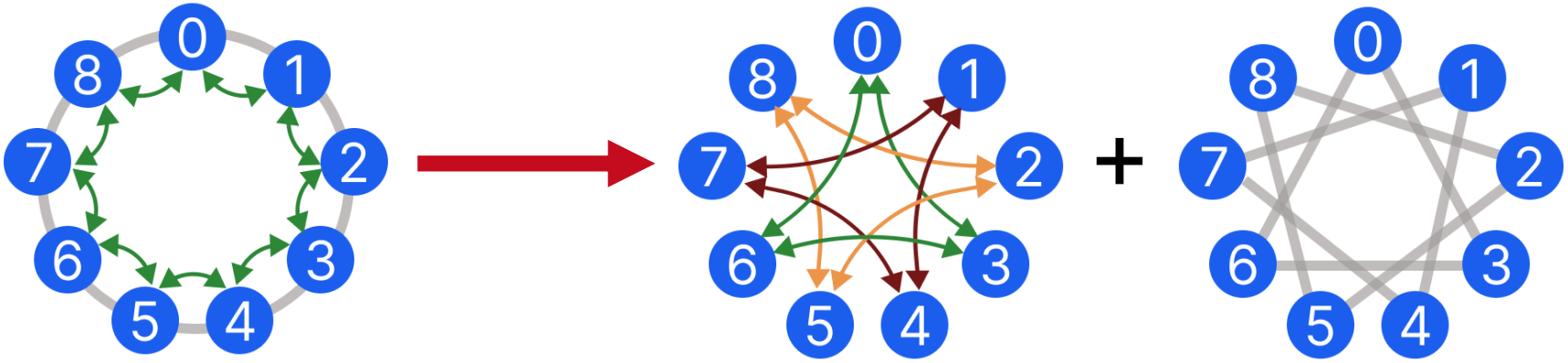
Phase 0 Phase 1

ReTri: Reconfigurable Trivance

- Recently proposed algorithm to reduce phases ⇒ Trivance
- Phase k: node u sends to $u + 3^k$ and $u - 3^k$
- Two peers per phase ⇒ matches the two optical links per node
- Symmetric exchange ⇒ both directions per optical link are used
- All-to-All completes in $\log_3 n$ communication phases



Case: Reconfiguration

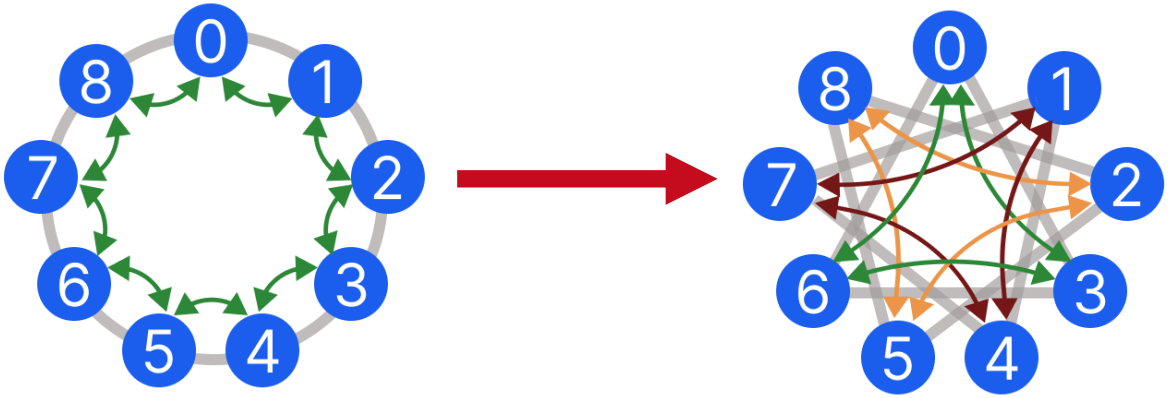


Phase 0 | Communication pattern | Topology reconfiguration

ReTri: Reconfigurable Trivance

- Recently proposed algorithm to reduce phases ⇒ Trivance
- Phase k: node u sends to $u + 3^k$ and $u - 3^k$
- Two peers per phase ⇒ matches the two optical links per node
- Symmetric exchange ⇒ both directions per optical link are used
- All-to-All completes in $\log_3 n$ communication phases

Case: Reconfiguration



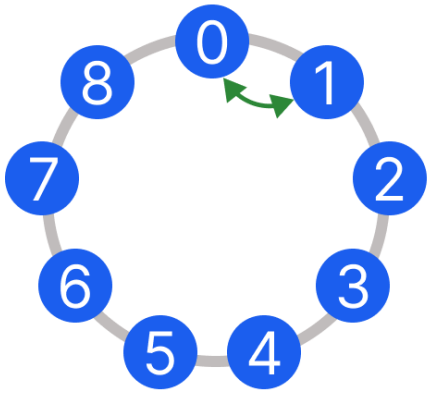
Phase 0 Phase 1

Minimal communication cost

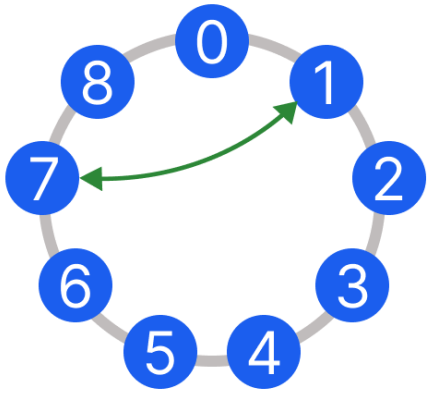
ReTri: block propagation

- In phase k, each block moves $+3^k$ or -3^k or stays local
- Each source-destination pair has unique power of three representation

Block propagation from node 0 to node 7:



Phase 0



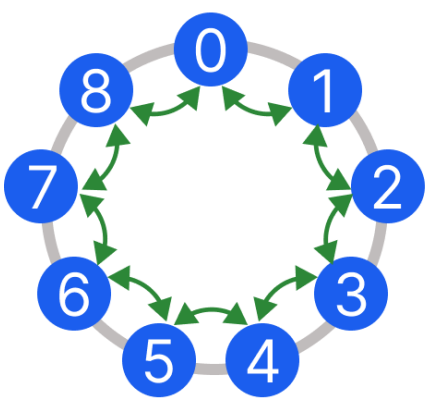
Phase 1

ReTri: ternary subrings

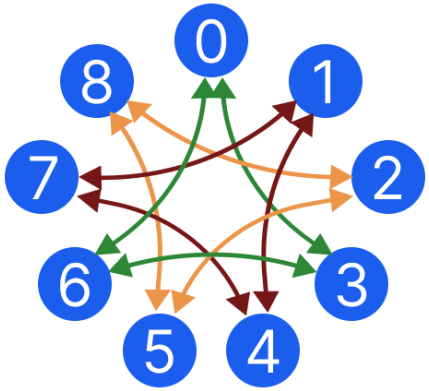
- In phase k, each block moves $+3^k$ or -3^k or stays local
- Each source-destination pair has unique power of three representation

- Peers in phase k are $u \pm 3^k \Rightarrow$ same residue class modulo 3^k
- Later offsets are multiples of $3^k \Rightarrow$ future peers are in the same subset

  $\Rightarrow$ partition in 3^k subrings of $\frac{n}{3^k}$



Phase 0

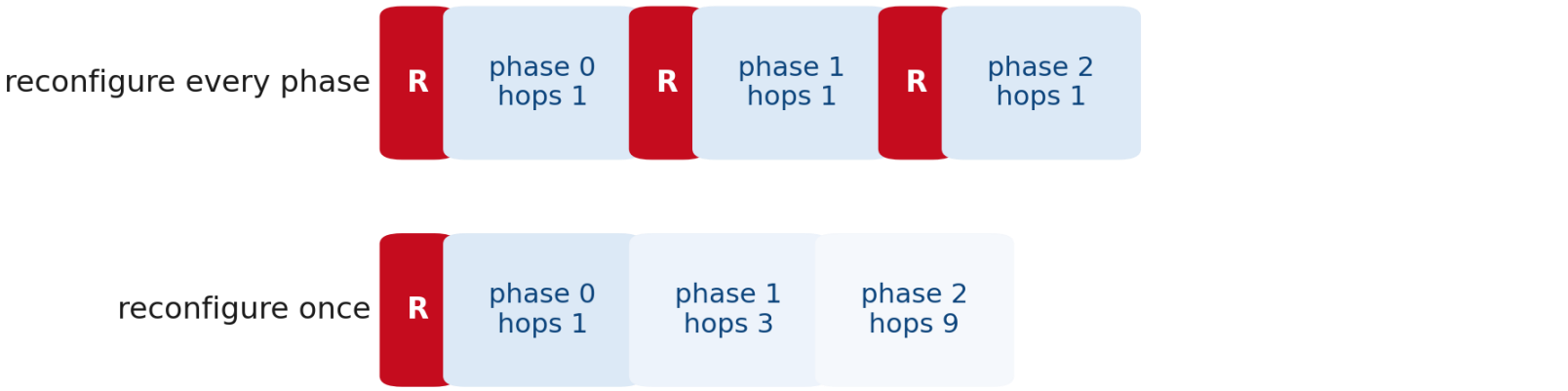


Phase 1

Reconfiguration: reuse instead of repeat

- After reconfiguring for phase k, both peers are directly connected
- Future peers remain inside the same subring: 3^{k+t} hops become 3^t hops

  ⇒ one reconfiguration can serve several phases
- ReTri needs $\log_2 3 \approx 1.58\times$ fewer phases than Bruck

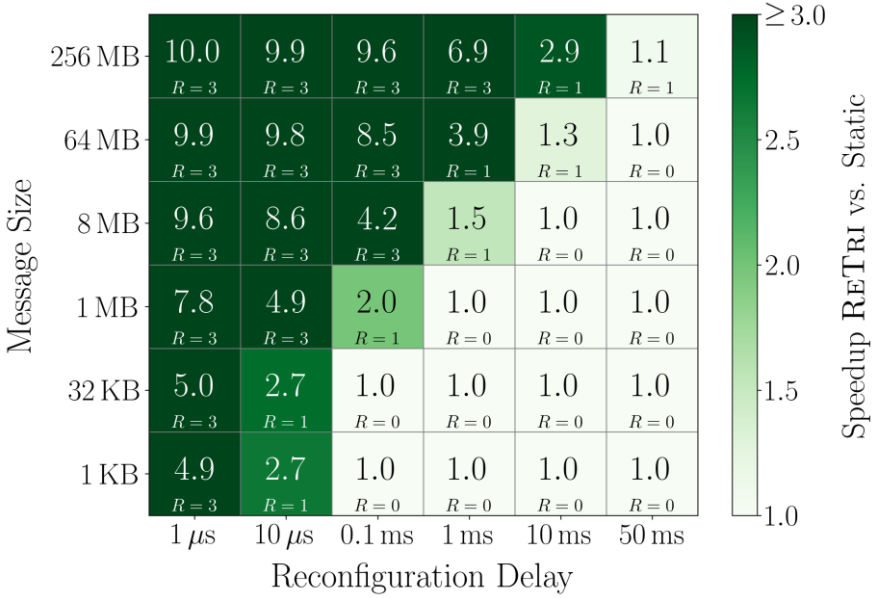


each skipped reconfiguration triples the distance | each reconfiguration costs δ

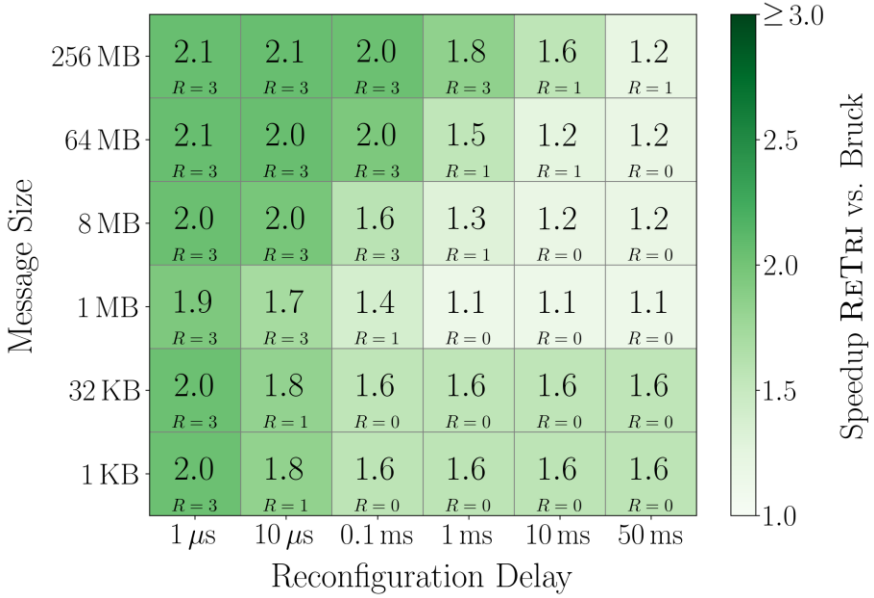
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Trivance vs. Static



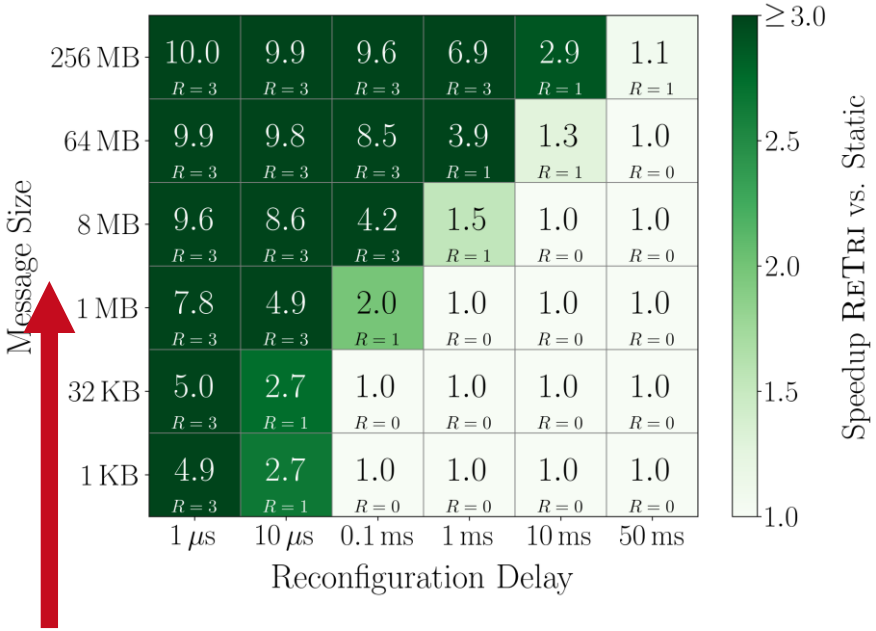
Trivance vs. Reconf. Bruck



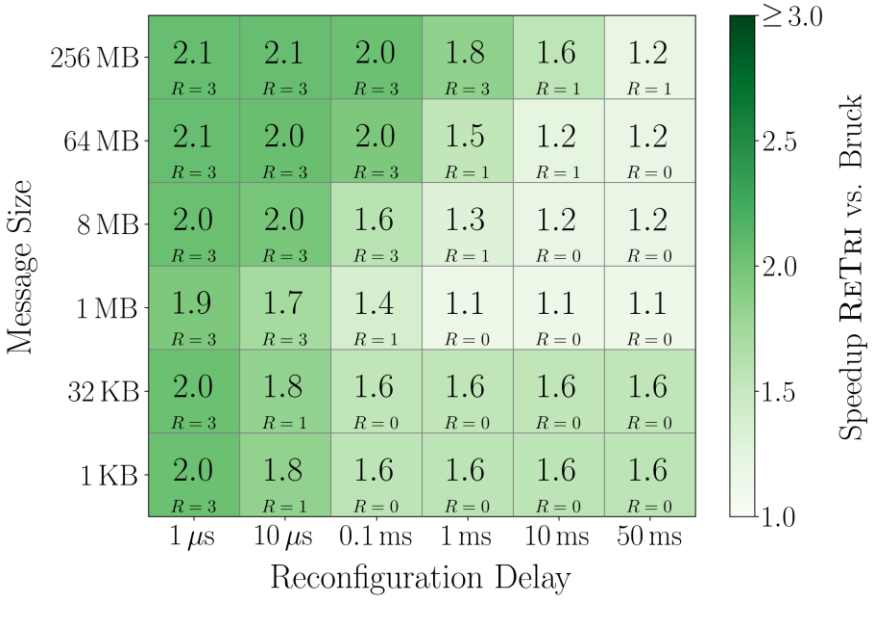
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Trivance vs. Static



Trivance vs. Reconf. Bruck

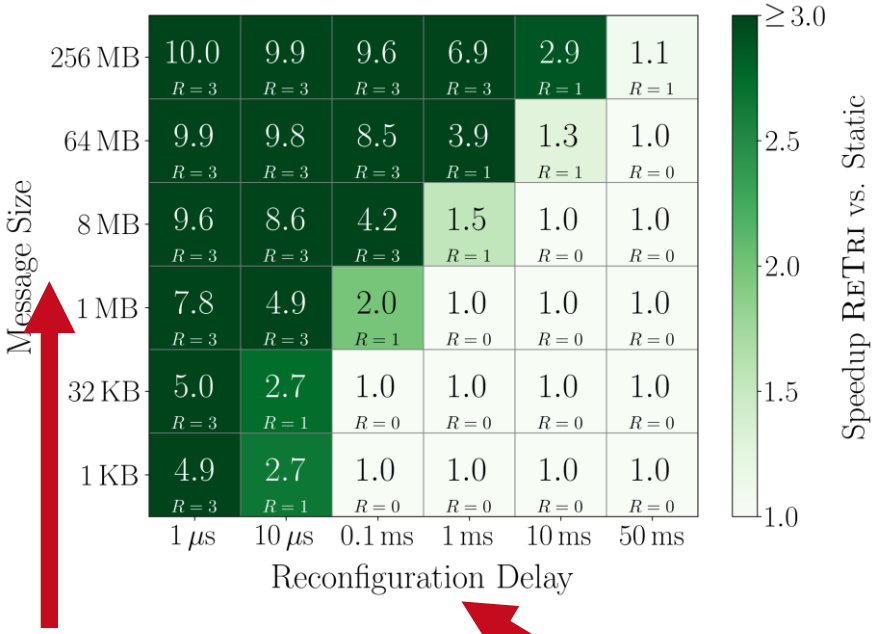


Speedup increases vs. Static with message size

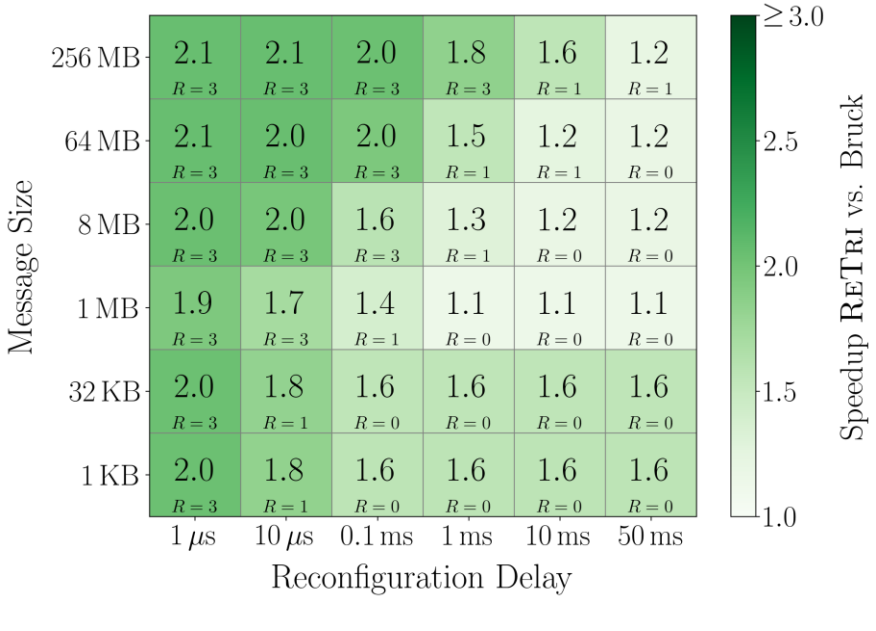
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Trivance vs. Static



Trivance vs. Reconf. Bruck



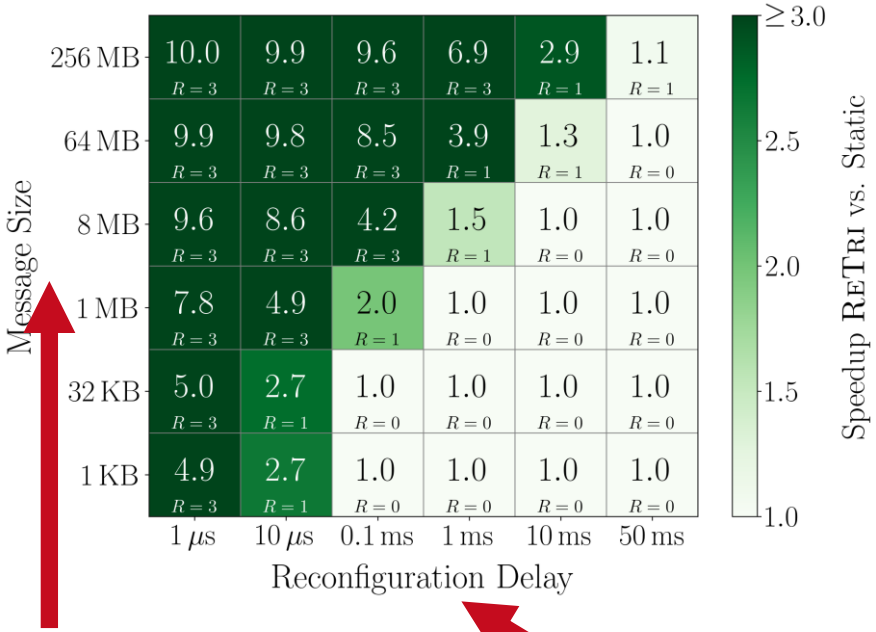
Speedup increases vs. Static with message size

Higher δ ⇒ fewer reconfigurations

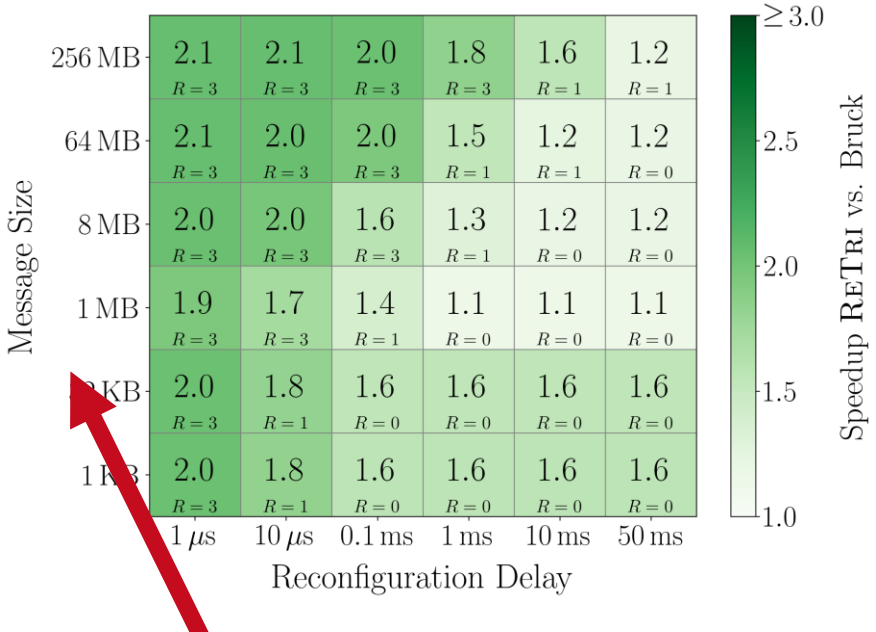
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Trivance vs. Static



Trivance vs. Reconf. Bruck



Speedup increases vs. Static with message size

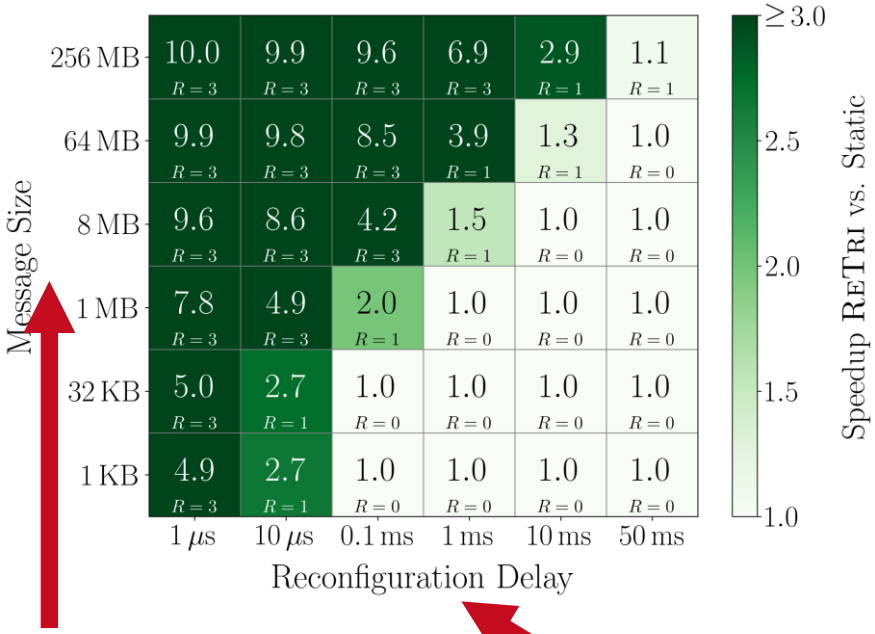
Higher δ ⇒ fewer reconfigurations

Improvement is constant

Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Trivance vs. Static



Trivance vs. Reconf. Bruck



Speedup increases vs. Static with message size

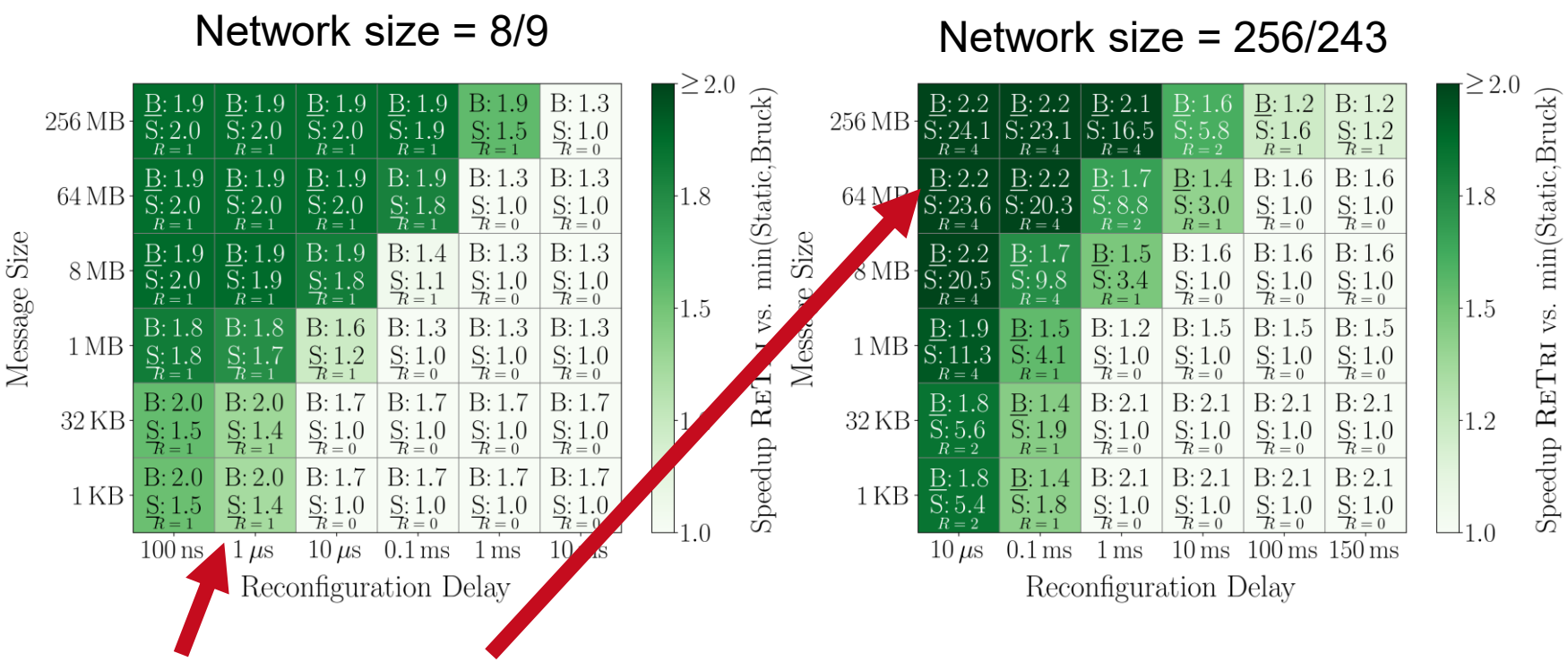
Higher δ ⇒ fewer reconfigurations

Improvement is constant

Without reconfigs. ReTri still improves Bruck

Simulation with Astra-Sim/ns-3

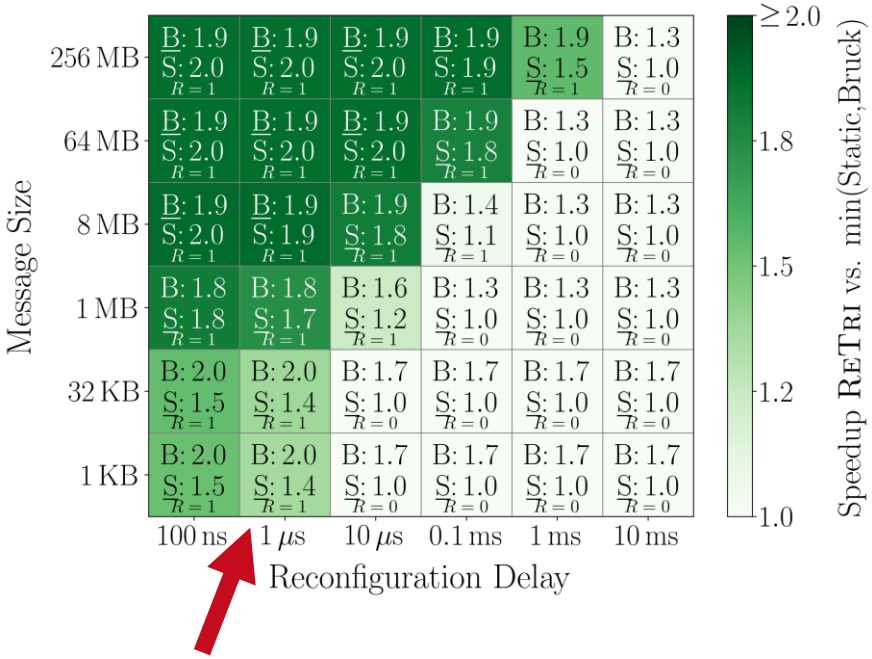
Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64



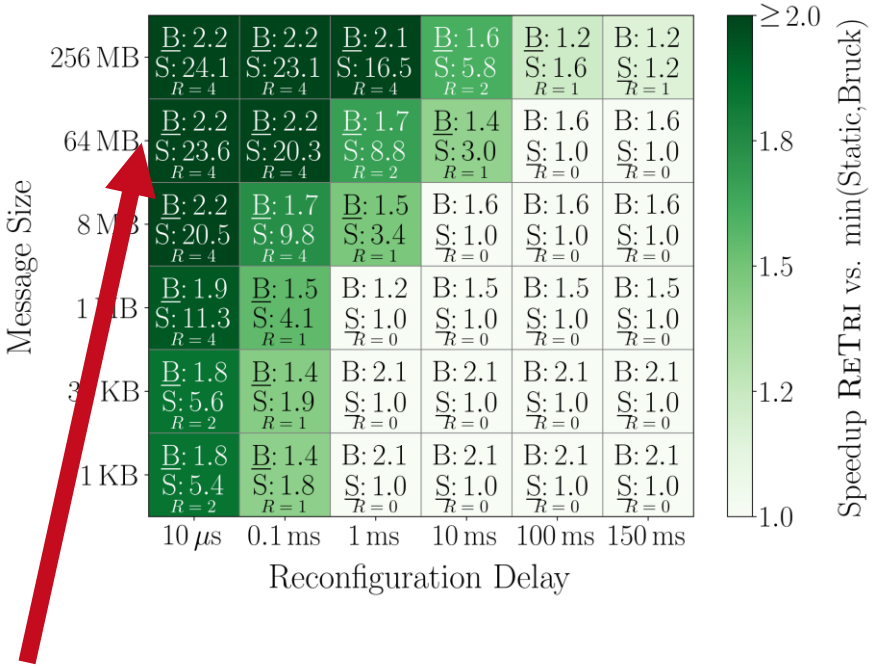
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Network size = 8/9



Network size = 256/243



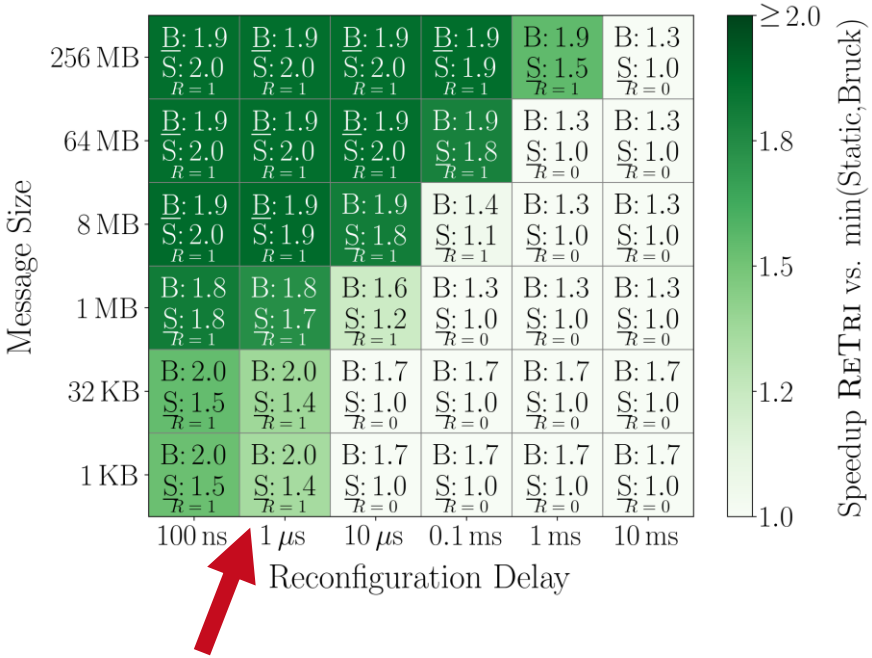
Continuous improvement over Bruck by 1.7-2.2

Larger networks = larger speedups

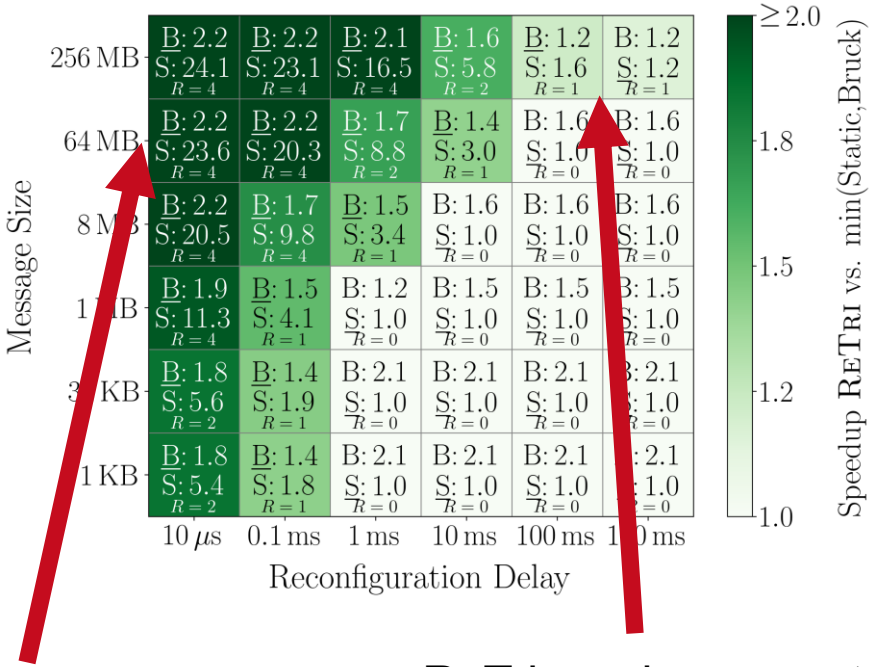
Simulation with Astra-Sim/ns-3

Astra-Sim + ns-3 · 400 Gb/s links · 1 µs link latency · 1.7 µs per phase latency · Messages 1 KB – 256 MB · δ = 100ns – 50 ms · ReTri n = 81 vs. Bruck n = 64

Network size = 8/9



Network size = 256/243



Continuous improvement over Bruck by 1.7-2.2

Larger networks = larger speedups

ReTri can improve static All-to-All for 256MB at δ =100ms

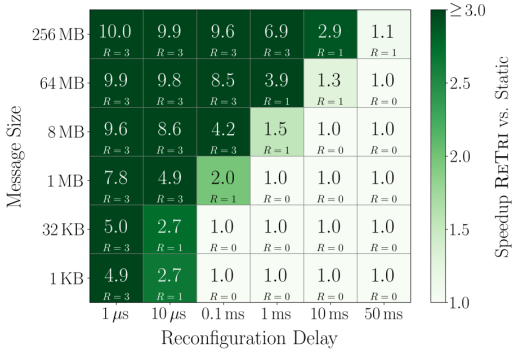
Summary

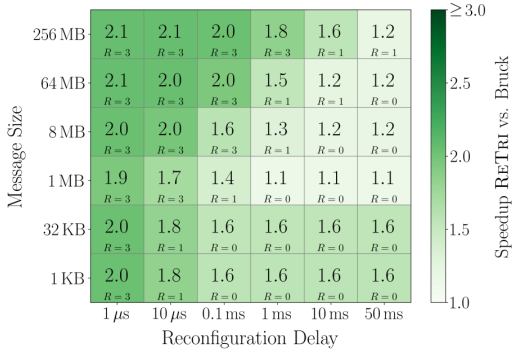
Objectives for collectives in reconfigurable networks:

⇒ algorithms need to follow constrains of reconfigurable networks

⇒ reduction of required communication phases

⇒ reusable reconfigurable links





Summary

Objectives for collectives in reconfigurable networks:

⇒ algorithms need to follow constrains of reconfigurable networks

⇒ reduction of required communication phases

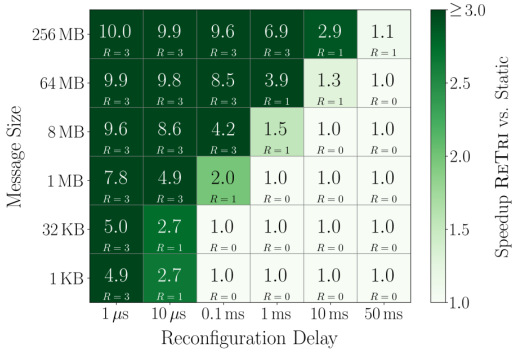
⇒ reusable reconfigurable links

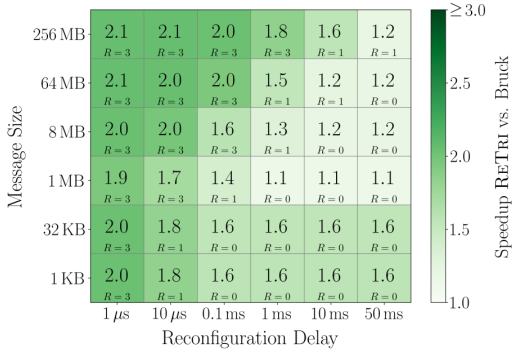
Reconfigurations during collective operations become desirable when:

⇒ Networks increase in size

⇒ Message size increases

⇒ Reconfiguration delay decreases





Summary

Objectives for collectives in reconfigurable networks:

⇒ algorithms need to follow constrains of reconfigurable networks

⇒ reduction of required communication phases

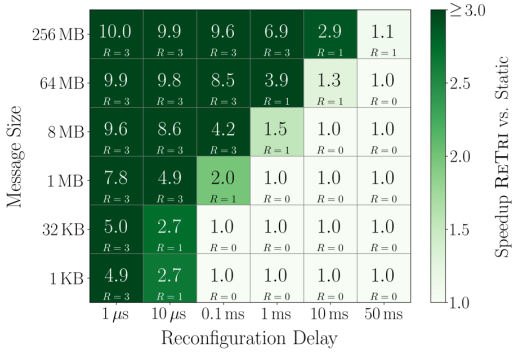
⇒ reusable reconfigurable links

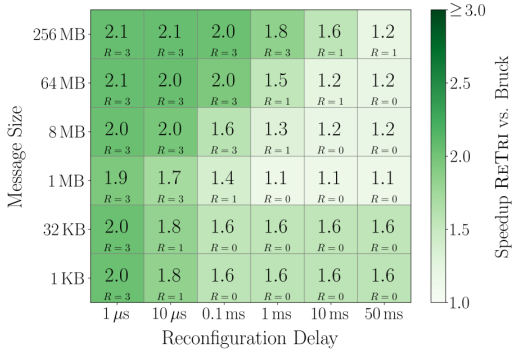
Reconfigurations during collective operations become desirable when:

⇒ Networks increase in size

⇒ Message size increases

⇒ Reconfiguration delay decreases





Thank you!

Email: juerss@campus.tu-berlin.de